



2º CUATRIMESTRE

ACTIVIDAD 15: DESARROLLO DE APLICACIÓN WEB - SIMULACIÓN DE CARRITO DE COMPRAS - PWD 7º 2º Grupo B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026

Estándar Internacional: e-Commerce & State Management in Vanilla JS

1. Introducción y Contexto

Cuando usas una página web como Mercado Libre o PedidosYa, la pantalla necesita "recordar" qué elementos vas seleccionando sin necesidad de recargar la página entera cada vez que hacés un clic.

A esa memoria temporal de la aplicación la llamamos **Estado (State)**.

Conceptos Fundamentales:

- **El Estado (State):** Es una variable (usualmente un arreglo [] o un objeto {}) donde guardamos los datos actuales en la memoria del navegador.
- **Reactividad en Vanilla JS:** Significa que cada vez que agregamos o quitamos algo del arreglo de datos, llamamos a una función que "vuelve a dibujar" (renderizar) la pantalla automáticamente para mostrar los datos actualizados.
- **Persistencia con localStorage:** Permite guardar esos datos en el disco del navegador para que, si el usuario presiona F5 o cierra la pestaña, la lista guardada no se pierde.

2. Conceptos Técnicos Clave

A. ¿Cómo se calcula el Total?

Matemáticamente, el total **T** de una lista de elementos es simplemente la suma individual de los precios de cada ítem agregado:

$$T = \text{Precio}_1 + \text{Precio}_2 + \dots + \text{Precio}_n$$

Si cada elemento tuviera una cantidad **q**, multiplicamos el precio unitario **p** por su cantidad **q**:

$$T = \sum_{i=1}^n (p_i \times q_i)$$

(Nota: El símbolo Σ simplemente significa "sumar uno por uno todos los elementos de la lista".)

En JavaScript, en lugar de usar un bucle for tradicional, usamos el método `.reduce()`, que recorre el arreglo acumulando los valores en una sola variable:

```
// Recorre el arreglo 'carrito' y acumula la suma de los precios arrancando desde 0
const total = carrito.reduce((acumulado, producto) => acumulado + producto.precio, 0);
```

3. El Desafío Técnico: Carrito de Compras / Selector Reactivo

Crea una solución de archivo único (index.html) que simule la selección de ítems y la acumulación en memoria local.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Actividad 15 - Carrito de Compras PWD 2026</title>
  <style>
    body { font-family: 'Segoe UI', sans-serif; background: #0f172a; color: #f8fafc; padding: 20px; }
    .grid { display: grid; grid-template-columns: 2fr 1fr; gap: 20px; }
    .card { background: #1e293b; padding: 15px; border-radius: 8px; margin-bottom: 10px; }
    .btn { background: #38bdf8; color: #0f172a; border: none; padding: 8px 12px; border-radius: 4px;
cursor: pointer; font-weight: bold; }
    .btn:hover { background: #0ea5e9; color: #fff; }
    .btn-danger { background: #ef4444; color: #fff; }
    .badge { background: #ef4444; color: white; padding: 2px 8px; border-radius: 12px; font-size:
0.8rem; }
  </style>
</head>
<body>

  <h1>&img alt="shopping cart icon" data-bbox="175 465 195 480"/> Módulo de Selección y Carrito de Compras</h1>

  <div class="grid">
    <!-- Catálogo de Productos / Opciones -->
    <div>
      <h2>Catálogo Disponibles</h2>
      <div id="catalogo"></div>
    </div>

    <!-- Resumen de Selección (Carrito) -->
    <div class="card">
      <h2>Mi Selección <span id="contador-badge" class="badge">0</span></h2>
      <ul id="lista-carrito"></ul>
      <hr>
      <h3>Total: $<span id="total-compra">0.00</span></h3>
      <button class="btn btn-danger" onclick="vaciarCarrito()">Vaciar Lista</button>
    </div>
  </div>

  <script>
    // 1. Base de datos simulada
    const productos = [
      { id: 1, nombre: "Sensor IoT de Flujo", precio: 4500 },
      { id: 2, nombre: "Módulo GPS Antirrobo", precio: 8200 },
      { id: 3, nombre: "Licencia Software Premium", precio: 12000 }
    ];

    // 2. Estado inicial (se recupera de localStorage o arranca vacío)
```

```

let carrito = JSON.parse(localStorage.getItem('pwd_carrito')) || [];

// 3. Renderizar Catálogo en pantalla
function renderizarCatalogo() {
  const contenedor = document.getElementById('catalogo');
  contenedor.innerHTML = productos.map(p => `
    <div class="card">
      <h3>${p.nombre}</h3>
      <p>Precio: ${p.precio.toFixed(2)}</p>
      <button class="btn" onclick="agregarAlCarrito(${p.id})">Agregar</button>
    </div>
  `).join("");
}

// 4. Agregar ítem al Estado
function agregarAlCarrito(id) {
  const producto = productos.find(p => p.id === id);
  carrito.push(producto);
  guardarYActualizar();
}

// 5. Guardar en localStorage y re-renderizar la pantalla
function guardarYActualizar() {
  localStorage.setItem('pwd_carrito', JSON.stringify(carrito));
  renderizarCarrito();
}

// 6. Renderizar Lista Seleccionada y Calcular Total
function renderizarCarrito() {
  const lista = document.getElementById('lista-carrito');
  const totalEl = document.getElementById('total-compra');
  const badge = document.getElementById('contador-badge');

  // Renderizar cada ítem de la lista
  lista.innerHTML = carrito.map((p, idx) => `
    <li>${p.nombre} - ${p.precio.toFixed(2)} <button
onclick="eliminarItem(${idx})">✖</button></li>
  `).join("");

  // Cálculo dinámico del total con .reduce()
  const total = carrito.reduce((acc, p) => acc + p.precio, 0);
  totalEl.textContent = total.toFixed(2);
  badge.textContent = carrito.length;
}

// 7. Eliminar un elemento por su índice
function eliminarItem(index) {
  carrito.splice(index, 1);
  guardarYActualizar();
}

// 8. Vaciar todo el carrito
function vaciarCarrito() {
  carrito = [];
}

```

```
        guardarYActualizar();
    }

    // Inicializar la aplicación al cargar la página
    renderizarCatalogo();
    renderizarCarrito();
</script>
</body>
</html>
```

4. Articulación con los Proyectos Integradores (Expo Técnica)

¿Su proyecto no es una tienda de comercio electrónico? **No hay problema.** El patrón de "agregar elementos a una lista temporal y guardarlos en memoria" se adapta a cualquier sistema:

- **Grupo 5 (Radar de Obra): Usar** esta lógica para agregar materiales de construcción de distintos corralones a un "**Presupuesto Final**" o cotización.
- **Grupo 6 (SafeBike) / Grupo 1 (Alpha-Sniper 97): Simular** el agregado de accesorios o componentes opcionales (alarmas, luces, soporte GPS, cámaras adicionales, etc.) antes de solicitar una instalación.
- **Grupo 7 (RAC) / Grupo 8 (TrashFlow): Adaptar** el carrito como una "**Lista de Reclamos / Materiales a Reciclar**" donde el vecino acumula varios reportes antes de presionar el botón "Enviar Todo al Municipio/Localidad".
- **Grupo 3 (EVA) / Grupo 4 (SIMC): Usar** la lista para acumular un historial de sensores y/o eventos seleccionados para generar un reporte agrupado.
- **Grupo 2 (Malvinas: Voces del Frente): Usar** la lista para acumular un historial de peleas contra la IA para generar un reporte agrupado o similar.

5. Recursos Oficiales para Investigar

- **MDN Web Docs (Mozilla):** [Guía de localStorage en JavaScript](#)
- **MDN Web Docs (Mozilla):** [Método Array.prototype.reduce\(\)](#)
- **W3Schools:** [Manipulación básica del DOM con JavaScript](#)

Frase del día: "Programar no es hacer magia onda Rada; es simplemente guardar los datos en una lista y volver a dibujarla/recrearla en pantalla cada vez que algo cambia." 🚀